

Reflective Report

The original idea for my project was to compare different methods of using Dual Device technology within Unity looking at different methods of showing coordinated views of the same space. The main design uses Unity's Multiplayer networking scripts to connect the two devices. I was then looking at different ways of sharing information between both devices and had two methods in mind. After some research I found that one method did not work, this was due to how Unity performed multiplayer synchronization. I was planning on using a Render Texture on the mobile with the PC sending image data to the mobile. Upon further research I found some code which claimed to do what I wanted to do. However the code had many errors and didn't work, I did some research to solve these error but could not find enough information.

I also asked a question on Unity Answers to see if anyone knew of a way to send a Render Texture over the network but got little help and mainly only questions asking why this was required. This method would have provided PC like quality on the mobile and would have had a constant send size, meaning the level could be very big. I have changed direction and am now looking to exploit as many features of the mobile and PC combo as possible.

I currently have:

Mobile and PC connected using Unity Multiplayer scripts

Connecting two devices together using Unity3d is rather easy; you just start a connection with one line of code and connect to the IP address with another line of code. This simple method of connection is great unless we need functionality which required the server then distributing commands to the clients and vice-versa in which case it becomes a lot more complicated. Much of my time so far has been tied up in connecting both the PC and mobile device in a way in which they can be used together.

A Player that spawns randomly on spawn points

I have created a prefab which is a coloured cube, these can be placed anywhere and the game will search them by tag and then place the player randomly at one of them. This means the user is always spawning somewhere new which should make the game more like other FPS's. These spawn points have script on them which hides them when the game starts, this means you can move them in the editor and they don't display in the game.

AI Zombies which spawn randomly and attack the player

The AI spawns work in the same way the player spawns work however these also have script to determine what can spawn at them, because I will have zombies and soldiers I can choose to spawn just zombies, just soldiers or both at a spawn point. A fixed amount of enemies will spawn at these points and whenever an enemy is killed a new enemy will be spawned.

The AI code is broken down into base and sub classes which handle the AI, a basic class called Actor is used by the AI and by the Player which stores health and also stores debugging code which displays debug information above the AI and player. The Actor class is then extended by a class (NPCBase) which defines some states and a simple state machine. The state machine has a function to change states and calls a function for each state which are then overridden by sub-classes to control different behaviours. The NPCBase class also contains AI logic code for looking at the player , following the player and more. The NPCBase class is then overridden by different NPC classes such as Zombie and Soldier which then override the state functions required and provide logic.

Mobile can follow the player or be on a Un-manned Aerial Vehicle (UAV) circling the level

The camera in the world has a script on it which allows it to be connected to any object in the scene, the camera will be given the same position and orientation as the object it's attached to. I've created a small UAV rotating around the centre of the level using a simple script. The UAV has a block that is oriented towards the centre of the level, a button on the mobile interface allows the UAV to be connected to the UAV. The Player also has a block attached to it which the camera can be attached to using a button, the current mobile controls allow the user to switch between 3rd person and UAV mode. I did have a problem with the camera lagging behind the player which I've now solved by running the camera movement script in a late update and not just a normal update.

The UAV script simply rotates using the Rotate function and it is also checked to make sure only the server runs the rotate class, if the client also ran this script the UAV would jump back and forth.

```
if(Network.isServer){  
    this.gameObject.transform.Rotate(0,0.1f,0);  
}
```

Mobile controlled Missiles

Using a button on the mobile I can create a missile that originates at the UAV's current position and travels towards the ground until it hits something. The missile uses accelerometers to move left and right which allow it to be aimed at targets. It uses the camera attachment script as well so the mobile camera follows it, the mobile camera will follow it until it's destroyed at which point the camera moves back to the UAV.

The Missile movement script lets the mobile move the missile using the devices accelerometers, the code below moves the missile in the y axis, I have yet to test this code but hope to have it tested very soon. I wrote the code using some guidance from an online tutorial for using the accelerometers.

```
float movey = 0;  
if(Mathf.Abs(Input.acceleration.y) > m_iMoveThreshold)  
{  
    movey = Mathf.Sign(Input.acceleration.y) * m_iSpeed;  
    transform.Translate(movey,0,0);  
}
```

What I need to add:

Mobile controlled object slots

The mobile view contains a menu down one side formed of buttons, these buttons will be used to start using different mobile controlled objects, such as the missile or sentry gun. At the moment buttons are used to simply call a function however the game will require more complex controls which will make the original buttons very complex in code, using slots means weapons can be allocated to slots and will be removed upon expiration. The slots will be given an object and will stay active until the object no longer exists, either a timer or an event destroys them. The buttons will also use images with text and not just plain simple text buttons.

Mobile controlled Sentry gun

The mobile will create a sentry gun from a prefab near the players location, the sentry gun will have a script on it which allows the mobile to control it by touching the screen and using the touch location to aim the sentry gun within a field of view. The camera will be attached using the camera attachment script like before.

Mobile controlled Ground Drone

This will act a lot like the sentry gun but will be controlled by the mobile device so it can move around the level. Another version of this might also fly and will work like a free roaming UAV with a machine gun attached.

Player has a weapon

The PC player currently has no weapon, the weapon will be created by adding layers to the player as well as a model. The weapon will use ray casting to hit things which will react as required.

Waypoint for the AI to follow

The AI will find the nearest waypoint to them and then follow them, the waypoints will be nodes placed in moveable areas, the AI will randomly move between waypoints within range of the other waypoints. This should allow the AI to search the level looking for the player and when alerted leave the waypoints and attack the player as required.

AI Soldiers that attack AI Zombies and Player

I have the basic code form a soldier but need to tell them to fire at the player when in range and also add some randomness to their actions so they don't fire when in immediate view of the player.

A level with some buildings and terrain

Creating a level with some complexity like any FPS should provide more interaction and will be the main reason a user is hooked when playing the game. This is vital to an FPS and needs to exist in some form; this won't aim at looking nice but rather just providing obstacles which the AI and player can navigate.

My Progress

The network connection script is used to connect two devices, the PC displays a button which says Play, this can be removed in the final artefact and is needed purely for testing. Once the Play button is pressed a server is created and an IP address is displayed in the corner of the game. The mobile displays a Connect button with a text field where the user can write the IP address as displayed on the PC, once the IP is written the Connect button will connect the Mobile to the PC. If the connection is successful the Mobile will display the level from the UAV, if it fails the mobile will fall back to the main screen after a few seconds. The connection script determines the PC and Mobile apart and then runs a script unique to each. These scripts determine what the PC can do and what the Mobile can do and are not available to the other platform.

The camera attachment script stores a reference to a game object in the AttachedTo variable which will update the camera to the same position and rotation as the object. The script is run in a LateUpdate which means the camera view is updated after the movement of everything in the scene has happened, this removed a noticeable lag when anything moves in the scene.

```
void LateUpdate()
{
    if(null != AttachedTo)
    {
        transform.position = AttachedTo.transform.position +
            new Vector3(m_fXOffset,m_fYOffset,m_fZOffset); // Set the Position
        transform.rotation = AttachedTo.transform.rotation; // Set the Rotation
    }
} // Run Every Frame
```

Below are some examples of my progress so far and screenshots of different aspects of the game with captions.

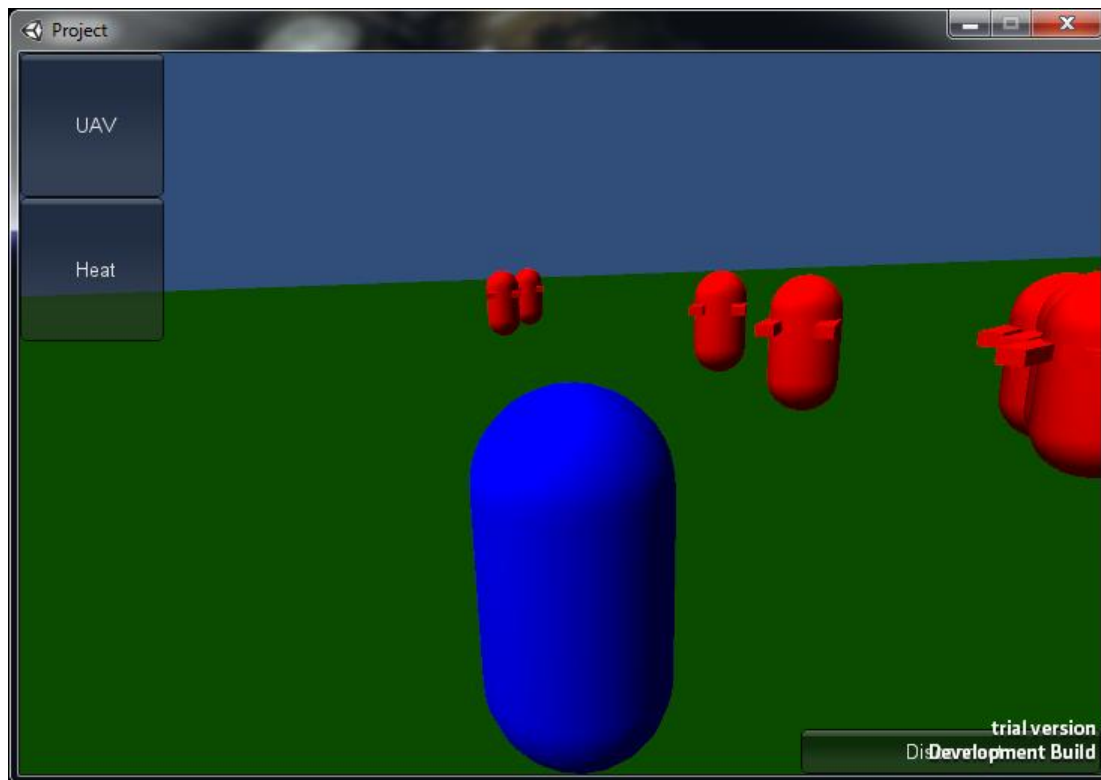


Figure 1 Mobile view in 3rd Person view



Figure 2 Mobile view in UAV mode also showing the menu screen.

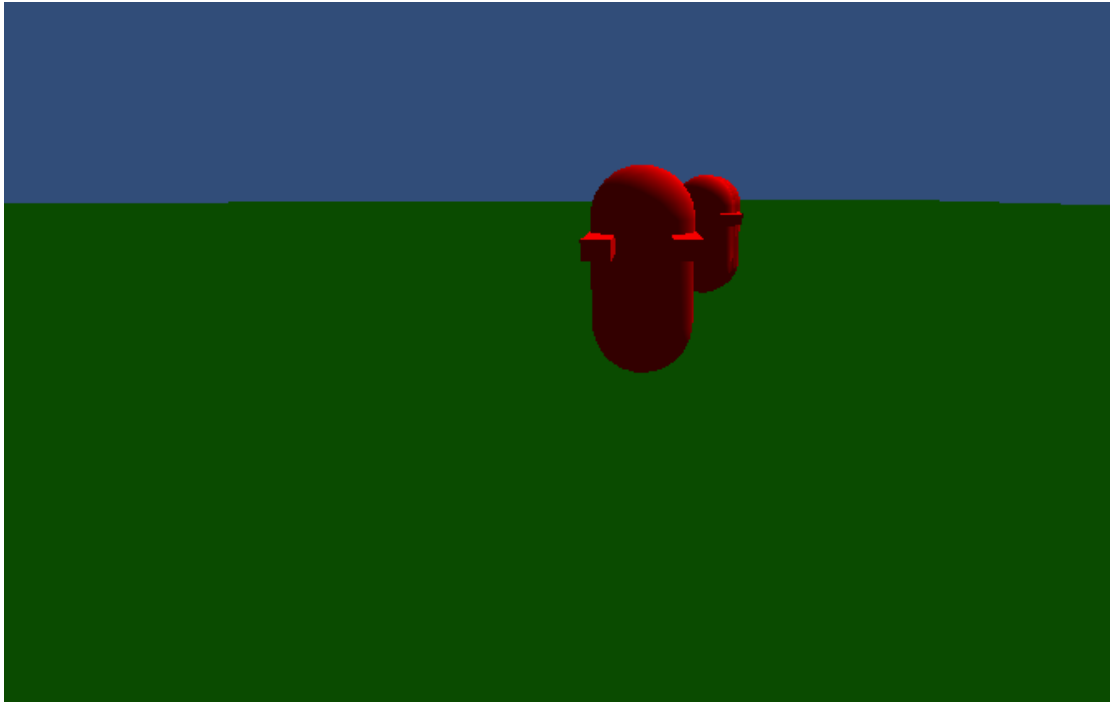


Figure 3 PC FPS view, the red things are Zombie AI.

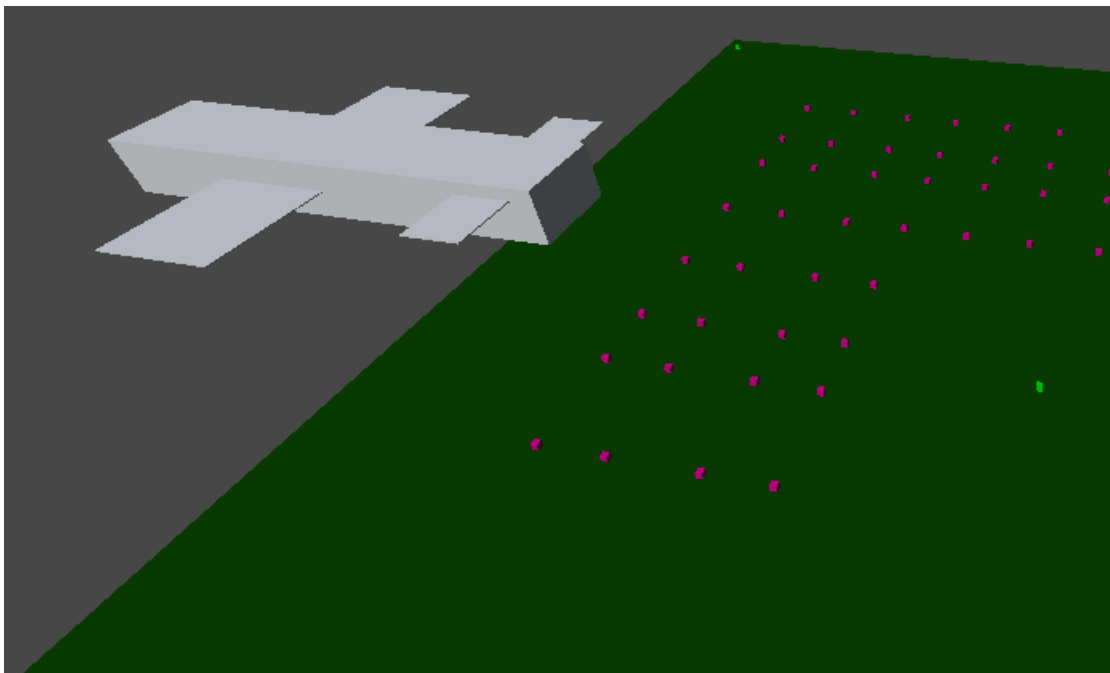


Figure 4 PC Unity3d scene view of the placement nodes for the AI (Pink) and player (Green).

I have a lot more work to do and have spent around 8 hours a week on this, the project hasn't run smoothly with a couple of weeks wasted looking at different connection methods which I also wrote documentation for. I have changed direction a little from my original plan and now aim at exploiting as many features of dual device gaming as possible. I would love to allow the user to control the FPS using the touch screen on the mobile, a sort of remote gamepad but this would mean my other features didn't work, I might build a small demo to example this with something in game. I am still using my Gantt chart as a guide but the times are very unrealistic, I am now addressing this by using it as a Work Breakdown Structure (WBS) than a time based plan. I have been writing some documentation whilst building the artefact, I have also started writing the final report.

Final Report Contents

Title

Contents

1.0 Introduction

1.1 Principles of Dual Device Gaming?

Dual Device (sometimes dual screen) gaming typically uses two computers with their own screens working together to display information about a single scene. The Wii U made by Nintendo utilizes Dual Device technology to connect a hand held gamepad which contains a screen and controls to a base unit which displays content on a TV. The TV and gamepad can show different information about a game which range from inventories to direct interaction with the actual game showed on the TV.

1.2 Project Objectives

This project explores the idea of using Dual Device gaming as a means to providing more interaction with a game beyond the typical constraints of a keyboard and mouse. The idea of Dual Device gaming is relatively new and this project aims at using existing hardware to provide an improved experience for the user.

The main focus of research in this project is based on Dual Device gaming which is primarily formed of two aspects, input and display. Dual Device gaming allows for multiple forms of input to be used together. This project looks at different input methods and how they could be combined to produce a more effective input system. Dual Device gaming also provides multiple coordinated views of the same space and this project looks at how different perspectives of the same information can be used.

The artefact is written using Unity3d which is a 3D game engine capable of creating First Person Shooters (FPS) for a range of devices including PC's and mobile devices supporting Android or iOS. Unity3d simplifies the process of making 3D games as well as having support for a number of useful features for this project. The artefact is built in steps looking at different aspects of development from connectivity between devices to the FPS and adding additional features.

2.0 Literature Review

This section contains content from the Contextual Report including most of the contents originally in the Literature Review and some additional research based on Unity3d. The Unity3d research leads nicely into the planning and design of the artefact section and helps the reader to understand some of the core concepts within the project. Below are the sub headings, these are the same as those in the Contextual Reports Literature Review.

- 2.1 Dual Device Technology
- 2.2 Multiple Views of the Same Space
- 2.3 Input Methods
- 2.4 Summary

3.0 Artefact

-3.1 Planning & Design

This section covers planning of the artefact covering connection, features and a general design of how the artefact works as a whole.

-3.2 Development

The Development covers creation of the artefact and problems encountered within development, this includes the connection problem, connecting two devices, mobile controlled items and keeping everything synced.

-3.3 The Artefact

This provides a look at the final artefact and how it all works and just clarifies the entire artefact. This section contains screenshots of the finished artefact a long with descriptions about how each part of the artefact works.

4.0 Evaluation

-4.1 Testing

This covers how the testing is carried out and what sort of testing is being used (Closed Box, Open Box, Functional and User Based).

-4.2 Self Evaluation

This section contains tests that I've performed against the final artefact compared to my expectations, plans and ideas. This includes testing in both closed and open box scenarios which covers a variety of factors that user testing doesn't cover.

-4.3 User Evaluation

Testing by users, these are gamers of a small selection which is where my artefact is aimed. Testing with non-gamers would mean they wouldn't be able to judge the impact of dual device within gaming. Users have been given a feedback form to fill in, the results of which are also discussed here.

5.0 Conclusion

-5.1 Conclusions

This will be used to judge if the project was a success, how dual device gaming could be used in the industry or not. This will also summarize user feedback, ease of implementation and my own feedback about the entire project, artefact and dual device gaming.

-5.2 Further Research

This will look at further areas of research for dual device gaming and further developments that could be made regarding my own project.

6.0 References

-6.1 References

This includes the references.